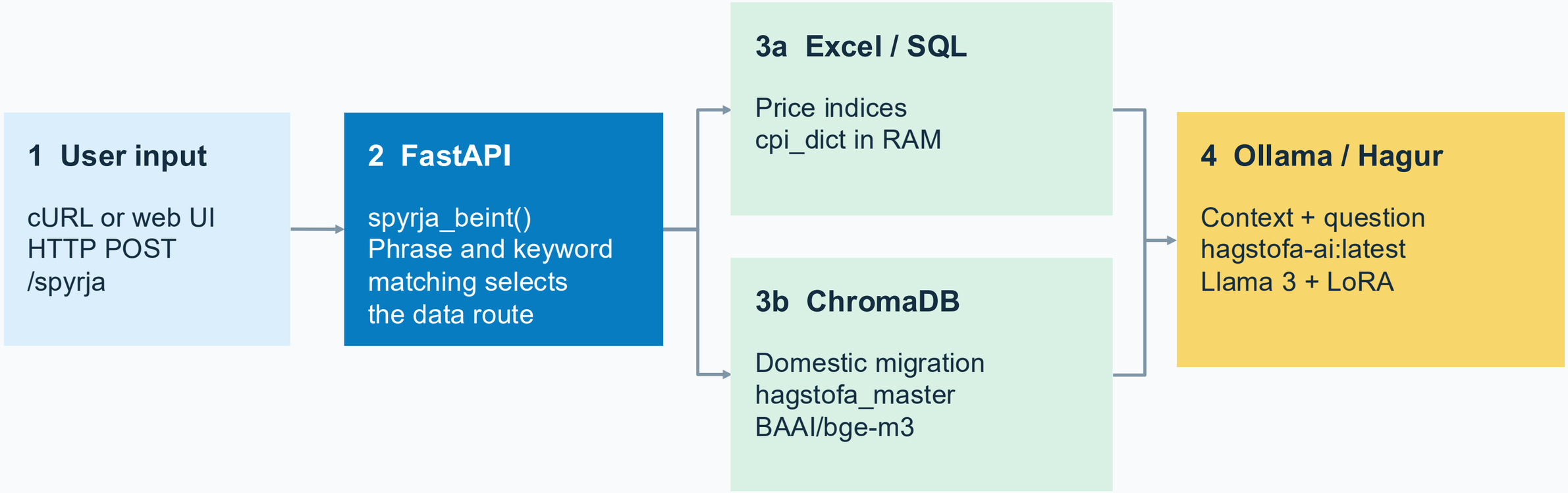
Track 1: Vector RAG and direct lookup

The question determines the data route. Hagur generates an answer from retrieved context.



Output: an Icelandic answer using values and context from the selected data route.

Track 1: Architecture layers and responsibilities

Data storage, query processing and answer generation have distinct responsibilities.

Layer	Main components	Responsibility
Data	ChromaDB persistent client Excel / SQL, cpi_df, cpi_dict METADATA_MIRROR	Loads data at startup. Caches index values and metadata in RAM. Retrieves text chunks using vector search.
Core processing	FastAPI and hreinsa_texta() threading.Lock() with ollama_lock:	Normalises text and selects the data route. Serialises Ollama calls within a single Python process.
Answer generation	Ollama /api/generate hagstofa-ai:latest temperature: 0.2	Combines the question with retrieved data. Low temperature reduces variability. Numerical validation is still required.

Concurrency: the lock does not span Uvicorn worker processes or prevent every out-of-memory error.

Track 1: Startup and query lifecycle

Metadata loads at startup. Retrieval and language models process incoming questions.

01 **Server startup**

FastAPI initialises data and connections using lifespan.
Older code may use @app.on_event("startup").

02 **In-memory mirror**

load_metadata_mirror() reads ChromaDB in batches of 2,000
and loads Excel data. Years, regions and sex are cached.

03 **Query and response**

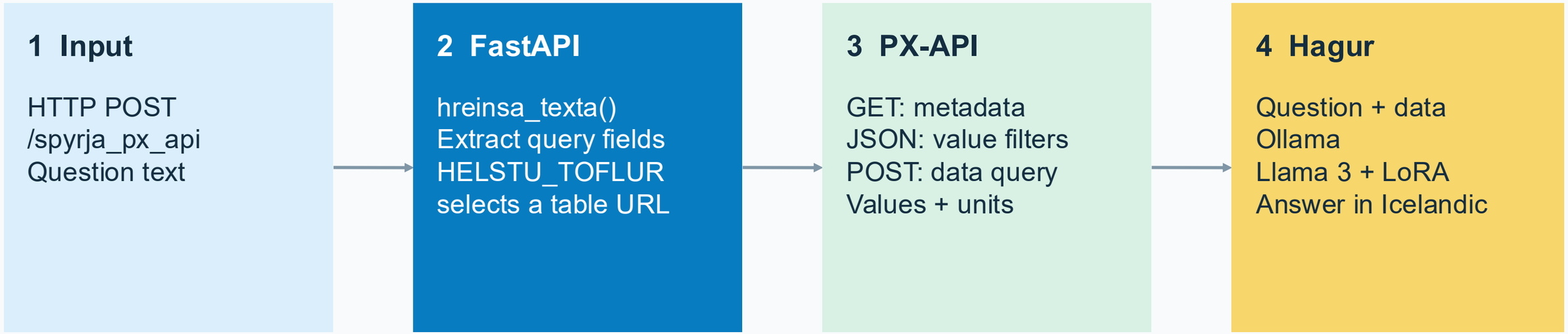
BGE-M3 embeds the query. ChromaDB retrieves text chunks.
Llama 3 tokenises the question and context to generate an answer.

Two distinct roles

BGE-M3: vector retrieval. Llama 3 + LoRA: generation from retrieved context.

Track 2: Direct data retrieval with PX-API

Target architecture without vector retrieval. Teams implement data access and Hagur integration.



PX-API returns official values. The backend validates selections, units and results.

LLM generation can still introduce errors. Include the source and selected dimensions in the answer.

Track 2: Execution flow in seven steps

The question becomes a validated dimension filter before the backend retrieves values.

01	**cURL POST**	The backend receives the question at /spyrja_px_api.
02	**Text normalisation**	Apply NFKD, remove combining marks and lowercase. Map þ/th and æ/ae explicitly **Normalization Form Compatibility Decomposition**. It is a Unicode method that.
03	**Table selection**	Match query terms and intent against HELSTU_TOFLUR in RAM.
04	**Metadata GET**	Fetch variable names, dimension codes and allowed values for the selected table.
05	**JSON filter and POST**	Extract years and regions, validate codes and retrieve the relevant data.
06	**Fallback handling**	Clarify invalid or ambiguous selections. Apply size limits to broader queries.
07	**Hagur generation**	Send the question, values, units and source together to Llama 3 + LoRA.

Track 2: A catalog of 2,009 tables in RAM

Planned scaling through a data-driven catalog and generic query code.

Startup loader

1. lifespan starts the loader.

2. Read hagstofa_catalog.json or .csv.

3. Validate the schema and build a RAM index.

4. Search the in-memory catalog per request.

No repeated disk reads for the catalog.
Network, search and LLM latency still apply.

Example schema for one table

```
{
  "table_id": "<table ID>",
  "title_is": "<Icelandic table title>",
  "api_url": "<verified PX-API URL>",
  "keywords": ["<keyword>"],
  "dimensions": {
    "<dimension code>": {
      "label": "<dimension label>",
      "values": {"<code>": "<label>"}
    }
  },
  "updated_at": "<ISO 8601>"
}
```

Add tables to the catalog. New dimension meanings or complex questions may require additional rules.

Team assignments: Track 1 vs. Track 2

Both tracks deliver their own web interface. The scope of backend work differs.

TRACK 1 VECTOR RAG

Provided starting point

FastAPI code, a price-index file and ChromaDB with domestic migration data.

Team tasks

Set up venv and test /spyrja with cURL.
Build your own chatbot web interface.
Connect the frontend to FastAPI.

RAG and Ollama integration are included.

TRACK 2 DYNAMIC PX-API

Provided starting point

A FastAPI starter for direct PX-API access and local setup.

Team tasks

Extend table search and GET/POST queries.
Support multiple questions in one API call.
Build Ollama integration and your own frontend.

Teams implement concurrency and responses.

Shared deliverables and developer setup

Both tracks deliver a working chatbot web interface connected to FastAPI.

Deliverables and testing

Your own web interface with clear loading and error messages.

Answers show values, units and sources where applicable.

Test with cURL and user questions in the browser.

Prerequisites

Data and models available. Ollama running with hagstofa-ai:latest installed.

Development setup

```
python -m venv venv
source venv/bin/activate

python -m pip install fastapi uvicorn requests \
  pydantic pandas openpyxl chromadb \
  sentence-transformers

uvicorn main:app --reload \
  --host 127.0.0.1 --port 8000
```

```
Windows PowerShell: .\venv\Scripts\Activate.ps1
Windows cmd: venv\Scripts\activate.bat
```